Machine Learning for Customer Churn Prediction

Nelson David Posso Suarez
Javier Alejandro Penagos Hernandez
Nicolas Castro Rivera

1 Executive Summary

Unsupervised learning provides techiniques useful for different problems that complement the projects. The re-creation of existing data, the anomaly detection or generate new data similar to existing data. Can be help in the domain of Churn Prediction, in special for working with a dataset like the KKBox selected for this project.

2 Motivation

The contribution of this workshop to the project, is implement unsupervised learning for strategy parts in the Customer Churn Prediction system. There is a problem with the unbalanced dataset, then the experiment will consist in explore the AE and VAE for show how good reply existing customers and then implement for generate customer churns.
Purpose of this new customers generated with real customer churns, is generate more data and get a better performace, that the models of w1 got. The experiment will conclude if this whether or not this is feisible depends on the results.

3 AE and VAE

Autoencoder and variational autoencoder are unsupervised neural network arquitectures

3.1 Data and preprocessing

The autoencoders reuse the tabular export built in Workshop 1 (version 5), where the *members*, *transactions* and *user_logs* tables are condensed into a single record of 59 features per customer (56 numerical and 3 categorical). The February cohort (992,931 customers, 6.39% churn) is used for development and the March cohort (970,960 customers, 8.99% churn) as the held-out test set; 881,701 customers appear in both months.

The Workshop 1 split (stratified and hash-based on `msno`) is reused so that the same customers are used across workshops. The churn label is never fed to the networks; it is only used to select the training population. The AE and the VAE are trained on the 697,095 non-churn customers of the *fit* partition

and validated on the 116,183 non-churn customers of the *stop* partition, while the full March cohort, including churners, is used for testing.

A preprocessor is fitted on the training population only. Numerical features are imputed with the median, skewed features are transformed with a signed $\log(1+|x|)$, clipped to the 0.1% and 99.9% quantiles and standardized; one constant column was removed, leaving 55 numerical inputs. The categorical features (`city`, `registered_via`, `last_payment_method_id`) are mapped to vocabularies of 22, 6 and 34 levels and one-hot encoded inside the network, giving an input dimension of 117. An inverse transform maps reconstructions back to the original 59 columns. The AE and the VAE share the same fitted preprocessor.

3.2 AE Architecture

The autoencoder receives tabular input of dimension $d_{\text{in}} = 117$: 55 standardised numerical features concatenated with the one-hot encodings of the three categorical columns (`city`: 22, `registered_via`: 6, `last_payment_method_id`: 34).

Encoder. Two fully-connected layers with decreasing hidden units compress the input to the bottleneck:

$$\mathbb{R}^{117} \xrightarrow{W_1,\,\text{ReLU}} \mathbb{R}^{64} \xrightarrow{W_2,\,\text{ReLU}} \mathbb{R}^{32} \xrightarrow{W_z} \mathbb{R}^{z}.$$

Bottleneck. Three latent dimensions were evaluated: $z \in \{8, 16, 32\}$. A linear projection (no activation) maps the last hidden representation to the code vector $\mathbf{z}$, which is the only information passed to the decoder.

Decoder. The decoder mirrors the encoder with reversed hidden widths $[32, 64]$ and then branches into two output heads:

- *Numeric head*: a linear layer (no activation) reconstructing the 55 standardised numerical features.
- *Categorical heads*: one independent linear layer per categorical feature, producing un-normalised class logits (22, 6, and 34 dimensions respectively).

Loss function. The reconstruction loss combines mean squared error for the numerical part with the mean cross-entropy over all categorical heads, controlled by a scalar weight $\lambda = 1.0$:

$$\mathcal{L} = \underbrace{\text{MSE}(\hat{\mathbf{x}}_{\text{num}},\, \mathbf{x}_{\text{num}})}_{\text{numeric}} + \lambda \underbrace{\frac{1}{C}\sum_{c=1}^{C} \text{CE}(\hat{\mathbf{x}}_c,\, x_c)}_{\text{categorical}}.$$

Figure 1 illustrates the full encoder–bottleneck–decoder pipeline.

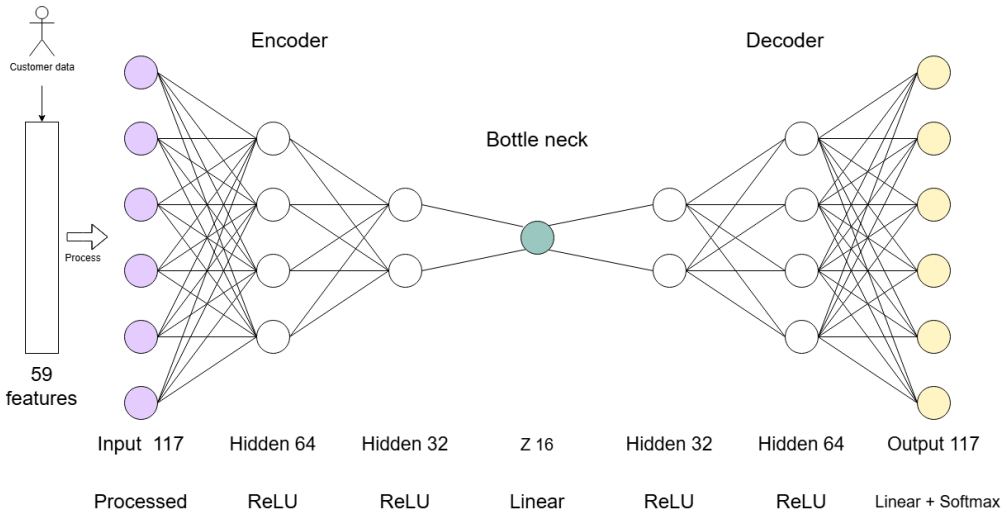


Figure 1: Standard Autoencoder architecture. The encoder compresses the 117-dimensional input to a z-dimensional code; the decoder reconstructs numerical features via a linear head and categorical features via per-class logit heads.

3.3 AE Train

All three latent configurations share the same training protocol. The Adam optimiser [1] is used with learning rate 10^{-3}, no weight decay, and a batch size of 512. Training runs for up to 50 epochs with early stopping (patience 5) monitoring the validation reconstruction loss. The model state corresponding to the lowest validation loss is restored at the end of training.

The training set consists of 697,095 non-churn customers from the February fit partition; the 116,183 non-churn customers of the stop partition are used as the validation set. The churn label is never seen during training.

Table 1 summarises the final train and validation losses for each bottleneck size. None of the three runs triggered early stopping; all completed the full 50 epochs.

Table 1: AE training results by latent dimension (epoch 50).

z	Train loss	Val loss	Train MSE	Val MSE
8	0.1274	0.1274	0.1250	0.1256
16	0.0443	0.0427	0.0433	0.0421
32	0.0161	0.0157	0.0156	0.0152

Reconstruction error drops sharply as z grows: z=32 achieves a validation loss roughly 8× lower than z=8, confirming that larger bottlenecks retain more customer-level information. Figure 2 shows the training and validation loss curves for all three runs.

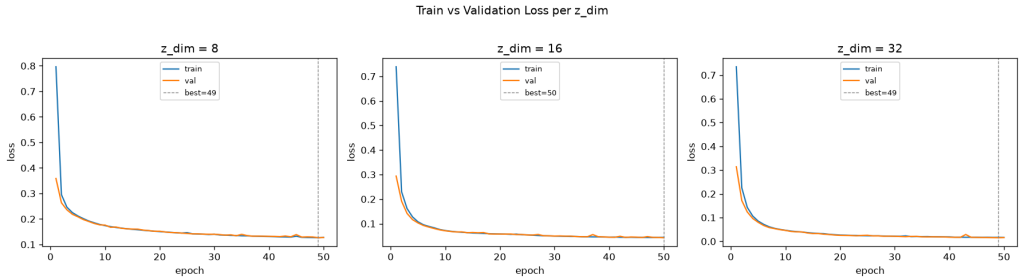


Figure 2: Training and validation reconstruction loss per epoch for $z \in \{8, 16, 32\}$. All curves converge smoothly without overfitting, and larger bottlenecks achieve consistently lower loss.

3.4 VAE Architecture

3.5 VAE Train

4 Reconstruction quality

5 Anomaly detection

6 Latent space visualization

7 Generate synthetic data

8 Analysis

References

[1] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.

[2] Rizvi, A. (2018). *WSDM – KKBox's Churn Prediction Challenge*. Medium. Retrieved from `https://asad-99rizvi.medium.com/wsdm-kkboxs-churn-prediction-challenge-d7c9cfa21cbd`